Memory-Side Perceptron-Based Prefetch Filtering

TRUNG LE*, University of Illinois at Chicago, USA
PEIYUN WU*, University of Illinois at Chicago, USA
ZHAO ZHANG, University of Illinois at Chicago, USA
ZHICHUN ZHU, University of Illinois at Chicago, USA

Data prefetching is a technique used by most processors to bridge the performance gap between processor and memory system in modern day computers. Effective data prefetchers hide the memory access latency by bringing the data into cache prior to its usage. However, inaccurate prefetches can bring unnecessary data into the cache hierarchy, consuming valuable bandwidth and resources, and polluting the cache.

In this paper, we introduce MSF, a Memory-Side perceptron-learning-based prefetch Filtering mechanism as an effective scheme to prevent cache pollution from inaccurate prefetches. Unlike existing prefetch filtering schemes that are solely based on core-side information such as prefetch accuracy, our scheme considers both core-side information and the memory state when making the filtering decisions, providing a holistic view of the system. It utilizes metadata available within the memory controller to learn memory access patterns, adaptively discards useless prefetches, and leads to increased accuracy for the underlying prefetcher. Our experiment results using octa-core workloads show that MSF can effectively reduce cache pollution, and increase the underlying prefetcher's accuracy, leading to an average performance increase of 7.0% and a memory energy saving of up to 39.3% compared to a baseline configuration with a state-of-the-art prefetcher. Furthermore, MSF can work with any generic underlying prefetcher.

CCS Concepts: • **Computer systems organization** → *Multicore architectures*; **Processors and memory architectures**.

Additional Key Words and Phrases: Prefetcher Filter, Memory Architecture, Perceptron Learning, Energy Consumption

ACM Reference Format:
Trung Le, Peiyun Wu, Zhao Zhang, and Zhichun Zhu. 2025. Memory-Side Perceptron-Based Prefetch Filtering. 1, 1 (July 2025), 22 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Memory wall is a problem that computer systems nowadays experience as the performance gap between the processor and memory system increases. Data prefetching is a technique used by most processors to bridge this gap. Effective data prefetching brings data into cache before it is requested by the core, virtually hiding the memory access latency. However, inaccurate prefetching can bring unnecessary data into cache that aren't used by the processor. Prefetches can be classified into two categories: 1) useful prefetches and 2) useless prefetches. Useful prefetches are referenced by the core at least once, while useless prefetches are evicted from cache without being referenced once. In our experiments running multiprogramming workloads with SPEC CPU [7] benchmarks using Signature-Path Prefetcher [18] in L2, on average, 62.7% (up to 94.7%) of the prefetched data are evicted without being referenced once.

*Both authors contributed equally to this research.

Authors' Contact Information: Trung Le, tle47@uic.edu, University of Illinois at Chicago, Chicago, IL, USA; Peiyun Wu, pwu27@uic.edu, University of Illinois at Chicago, Chicago, IL, USA; Zhao Zhang, zhangz@uic.edu, University of Illinois at Chicago, Chicago, IL, USA; Zhichun Zhu, zzhu@uic.edu, University of Illinois at Chicago, Chicago, IL, USA.

New Paper, Not an Extension of a Conference Paper.

Manuscript submitted to ACM

Inaccurate prefetches can negatively impact the performance, as they consume valuable bandwidth and resources that could serve demand requests and useful prefetches. Several schemes have been proposed to filter out unnecessary prefetches from polluting the cache hierarchy. The filtering technique can be either software-based [22], [35], or hardware-based [4], [6], [23], [38], [19], [34]. Hardware-based prefetch filters typically implement a filter engine directly in the cache hierarchy, acting as an independent check on the usefulness of incoming prefetches. However, those existing prefetch filtering schemes make the filtering decisions solely based on the core-side information such as the prefetch accuracy or age. We believe such decisions should be holistic and consider both the core-side and memory-side information. For instance, when the memory is underutilized or has good locality, prefetching filtering could be less aggressive; while when the memory is over-utilized or has poor locality, prefetching filtering should be more aggressive.

We introduce *Memory-Side perceptron-based prefetch Filtering* (MSF), an accuracy- and memory-state-aware prefetch filtering mechanism that can adaptively learn memory access patterns and predict whether an incoming prefetch is useful or beneficial. It filters predicted useless prefetches from further polluting the cache hierarchy and predicted non-beneficial prefetches from competing memory resources with demand requests. The filtering engine *Prefetch Analyzer Module* (PAM) is inserted directly in the memory controller, governing all Read requests coming to the memory system. PAM uses perceptron-learning that utilizes various features representing cache or memory status such as prefetch accuracy and memory page hit rate to adaptively predict the usefulness and benefit of any given prefetch request. If a prefetch is deemed useless, it is simply discarded from the Read queue. If a prefetch is predicted to be useful, then the request proceeds as a normal request to the memory access scheduler. A good prediction by PAM effectively prevents unnecessary prefetches from further polluting the caches and waste memory bandwidth.

We build a detailed memory simulator and integrate it into gem5 [5] simulator. The simulations run octa-core workloads, built with SPEC2006 and SPEC2017 [7] benchmarks using SimPoint3.0 [28]. The results show that MSF can effectively eliminate unnecessary prefetches from polluting the cache and significantly reduce memory energy consumption. Compared to the baseline with a state-of-the-art prefetcher, Signature Path Prefetcher [18], MSF improves performance by 7.0% on average (up to 18.4%) over 55 multi-programming workloads with SPEC benchmarks, and saves, on average, 10.6% of memory energy consumption (up to 39.3%) as the memory controller wastes less energy serving unnecessary prefetches. Furthermore, our experiments show that MSF works for any generic underlying prefetcher as it improves performances for all evaluated prefetchers. Compared to existing prefetch filtering schemes, MSF outperforms both PPF [4] and PADC [19] by 1.6% and 5.2% on average in memory-intensive workloads. Note that PPF methodology only works specifically for SignaturePath prefetcher, while MSF works well with any underlying prefetchers.

The rest of the paper is organized as follows: Section 2 discusses the background and related work, and Section 3 describes our proposed MSF scheme. Section 4 shows the experimental environment and the results are discussed in Section 5. Finally, we conclude our work in Section 6.

2 BACKGROUND AND RELATED WORK

2.1 Data Prefetching

Data prefetching is a well-studied technique that is deployed in most computer systems nowadays. The goal of prefetching is to preemptively fetch data into cache before it is requested by the processor, effectively hiding the memory access latency. Most prefetchers utilize metadata within the core, such as PC, request's tag, delta, and signature, to predict memory access patterns [18], [15], [10], [13], [26], [2], [37], [12], [1], [20]. However, inaccurate prefetches can impact the performance of the system negatively by causing cache pollution and increasing contention on memory

bandwidth [36]. Therefore, prefetcher schemes aim to increase accuracy while moderating their aggressiveness to prevent *over-fetching*.

2.2 Prefetch Filtering

Prefetch filtering has been proposed to filter out unnecessary prefetches. The filtering mechanism can be either software-based [22], [35], or hardware-based [4], [6], [23], [38], [24], [31], [14], [19], [34]. The hardware-based prefetch filtering mechanisms are typically deployed at core-side, and housed directly in the processor [38], or inside of the underlying prefetcher [4], [6], [23], [34]. Some deploys their filters inside the memory controller [14], [19]. However, most of the features examined by the above filters are already used in typical prefetchers, such as PC, branch history, and prefetcher's accuracy, lateness, and signature. The filters do not have new information to aid the filtering decisions, since they are already analyzed by the prefetchers. In addition, those filters do not consider the memory status when making the filtering decisions. In contrast, our proposed scheme MSF closely examines both core-side and memory-side metadata to provide a more detailed look of the application's access patterns, and utilizes information that aren't available inside the cache hierarchy such as page hit rate, ratio of demand/prefetch requests, and memory access hot-spots.

Of those existing prefetch filtering schemes, PPF [4] and PADC [19] are closely related to our work. *Perceptron-based Prefetch Filter* (PPF) uses a perceptron-learning engine, housed directly in the underlying prefetcher, to filter out unnecessary prefetches. However, PPF was designed to work with a specific prefetcher called *Signature Path Prefetcher* (SPP) [18], as five out of its eight perceptron features are derived directly using SPP's metadata. *Prefetch-Aware DRAM Controller* (PADC) inserts its filtering engine in the memory controller. It adaptively schedules the demand requests and prefetch requests with different priorities based on the accuracy and timeliness of the underlying prefetcher. PADC also drops old prefetches based on a simple *age* threshold. However, this simple filtering mechanism does not take other important information into account, such as the current prefetcher's accuracy and hot-spot areas in memory, nor can it recognize memory access patterns.

2.3 Perceptron Learning

Perceptron is a simple neural network containing trained weights to recognize correlations between its input and the pattern [9]. Perceptron learning was proven to work with various micro-architectural designs within the processor, such as cache management [17], [33], [3], [34], and branch prediction [16]. For example, *Hermes* [3] is a perceptron-based engine that predicts whether a load request will go off-chip. The module is placed in-core, and it analyzes all load requests made by the processor. If a request is predicted to go off-chip, the data is fetched directly from the off-chip memory in concurrent with accessing the cache hierarchy. However, Hermes introduces additional bandwidth usage due to additional memory requests generated by the Hermes engine. In contrast, MSF analyzes and rejects unnecessary prefetches coming into the memory hierarchy, reducing the overall memory power consumption.

2.4 Hashed Perceptron Learning

Our proposed MSF scheme uses the *hashed perceptron* [32] variation of the original perceptron design. In the original design, perceptron applies *dot-product* calculation with the weights, which is an expensive and long-latency operation. Hashed perceptron hashes the features into indices of the weight table and applies *sum* operation across the indexed weights. A binary decision is made if the sum crosses the activation threshold. Furthermore, training in hashed perceptron is a simple increment/decrement of the weight values. Compared to the original perceptron technique,

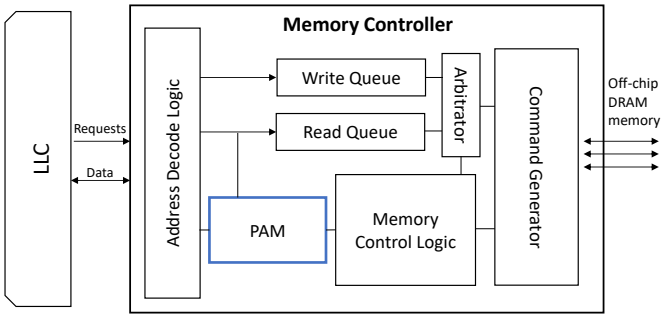

Fig. 1. Overview of MSF.

hashed perceptron significantly reduces latency of both the inference and training stages, as hashed perceptron does not utilize dot-product operator.

3 PROTOTYPE DESIGN OF MSF

3.1 Motivation

As we have discussed above, prefetch filtering is an effective technique that can filter out unnecessary prefetch requests generated by the underlying prefetcher and improve the system performance and energy-efficiency. However, existing hardware-based prefetch filtering mechanisms make the filtering decisions solely based on the core-side information such as the prefetching accuracy. Such kind of filtering decisions are not optimal since they ignore the memory status. For instance, when the memory system is under-utilized or the memory access pattern has good locality, the prefetch filtering could be less aggressive; while when the memory system is over-utilized or the memory access pattern has poor locality, the prefetch filtering should be more aggressive.

In addition, the core-side prefetch filtering techniques usually utilize the information that has already been analyzed by the underlying prefetcher in making the prefetch decisions, such as PC and prefetch accuracy. The filter does not add much new information to aid the filtering decisions. For example, *Perceptron Prefetch Filter* (PPF) [4] was designed to work with a specific prefetcher called Signature Path Prefetcher (SPP) [18]. Five out of its eight features are derived directly using SPP's metadata. This also poses as an obstacle to implementing a new prefetcher in the processor, since PPF needs to be retrained from scratch using the new prefetcher's metadata.

Our target is to design a prefetch filter that considers both the core-side and memory-side information when making the filtering decisions and works with any generic prefetcher. The filter will be placed in the memory controller and only use available metadata from the core and the controller. So any changes made to the underlying prefetcher's hyper-parameters (e.g., aggressiveness, confidence level, depth, etc.) will not affect the filter designs. In addition, since the memory controller is usually on-chip, the filter can receive feedback information from the cache with minimal overhead. Another advantage of placing the filter in the memory controller instead of in the cache as other filtering schemes do is that it can significantly reduce the storage overhead used by the filter. This is because a typical computer has many more cores than memory channels. Our proposed memory-side prefetch filter needs only one filter per memory channel, while core-side prefetch filters usually need one filter per core (cache).

3.2 Prototype Design Overview

Figure 1 shows a high-level overview of *Memory-Side perceptron-based prefetch Filtering* (MSF). A *Prefetch Analyzer Module* (PAM) is placed in the memory controller, overseeing all memory Read requests coming in from the Last-Level Cache (LLC). PAM uses perceptron learning to pull together features, derived from the available metadata within the memory controller and the processor, and determines the usefulness of a prefetch. As shown in our experiments in Section 5, MSF works with any generic underlying prefetcher, and can be used with neither hyper-parameter modifications to the underlying prefetcher nor any changes to the features selection.

There are two stages in MSF: *inference* and *training*. In the inference stage, PAM analyzes every prefetch request coming to the controller and determines whether or not the request should be served. Since prefetches are speculative read requests, not every generated prefetch will be consumed by the processor once data is returned from memory. The inference stage will discard any prefetch deemed useless, allowing the controller to focus on serving only imperative memory accesses. Typically, any requests coming in the memory controller is not immediately served, but rather enqueued in Read Queue/Write Queue, awaiting for the scheduling policy to choose the next memory access candidate. Due to this reason, the inference latency can be completely hidden from the request's critical path, as the decision would be constructed before it is served from the Read Queue.

PAM relies on the feedback from LLC and its internal logging tables to train and improve its decision-making module. PAM training is invoked when (1) a prefetched cache line is consumed in the cache hierarchy, or (2) a prefetched cache line is evicted from cache without being used, or (3) eviction from logging tables. Furthermore, these metadata are already accessible in-core and can be readily made available to the memory controller. Since the memory controller is integrated on board, retrieving LLC feedback incurs minimal overhead.

As the impact of PAM decision is not immediately known, two *logging tables* are inserted to store previous decisions made by PAM. The positive logging table stores the accepted prefetch requests, while the negative logging table stores the rejected prefetch requests. Each entry in the logging tables contains all the metadata necessary to retrieve the weight of each feature used. Once the feedback from LLC is received, the entry corresponding to the prefetch request is indexed and retrieved from the tables. Its weight values are incremented/decremented depending on whether the prefetched data was useful or not. The training latency can also be completely hidden as it does not happen on the request's critical path, but rather in parallel with the memory controller serving the request.

Unlike other filtering schemes that only examine core-side metadata [4] [19], MSF monitors both core-side and memory-side features that can convey information not been analyzed by the underlying prefetcher. All the features deployed in MSF's perceptron-learning mechanism are discussed in more detail in Section 3.8.

3.3 Prefetch Analyzer Module

Figure 2 shows the core components of Prefetch Analyzer Module (PAM). The *Feature Collector* is in charge of gathering all the metadata required to compute the features. All the features are hashed into their respective index values to the *Weight Table*. The weights are accumulated and fed into the Activator for the final decision. Once the decision is finalized, the metadata values are used to form an entry in the *Positive/Negative Logging Table*, details of each entry is shown in Table 4. All the evicted entries from logging tables are used as training opportunities, as discussed in Section 3.6.

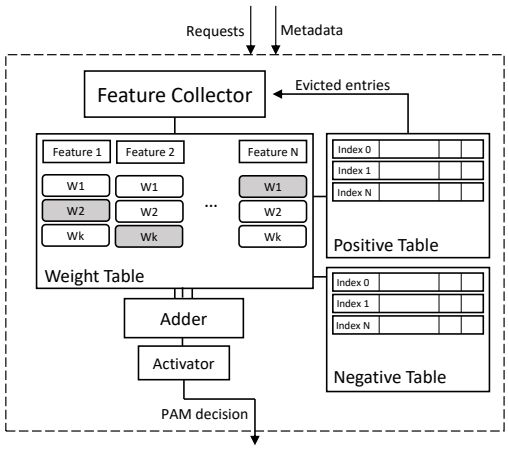

Fig. 2. Prefetch Analyzer Module design.

The PAM's main purpose is to observe and predict the usefulness of incoming prefetches from LLC. PAM self-trains its perceptron weights based on the feedback and evicted entries from the logging tables. Furthermore, the latencies of *inference* and *training* stages can be completely hidden from the request's critical path.

3.4 Inferencing

PAM inferencing is triggered on every memory read request coming into the controller. There are two types of requests: a speculative prefetch that is generated by the underlying prefetcher, and a true demand from the processor. PAM analyzes the prefetch and makes a binary decision on whether or not the prefetch should be accepted. If a prefetch is deemed unnecessary or not beneficial, it is simply discarded and the memory controller returns a signal letting the cache know the prefetch's *Miss-Status-Holding-Register* (MSHR) entry can be freed for subsequent misses. If the prefetch is determined to be useful or beneficial, then it can proceed as a typical request. Note that a prefetch request is upgraded to a demand request if there is a demand request whose address matches with the prefetch's address [19]. Once a prefetch is discarded, the memory controller returns a signal to drop the prefetch's entry in MSHR. However, if such prefetch request was upgraded to demand request in the meantime, MSHR sends a demand request once *'drop'* signal is received. This ensures all discarded prefetch requests have no true demand to the same address and MSF does not discard demand requests.

Since prefetches are speculative read requests in nature, its data might not be consumed once returned from the off-chip memory. A successful analysis prevents unnecessary prefetches from further polluting the caches and saves memory power consumption (details in Section 5). If the prediction is incorrect, meaning the speculative prefetch is going to be consumed by the processor but it is discarded by the controller, then a regular load request to the same address will eventually arrive. Therefore, a bad prediction by PAM will not stall the program execution, however it could negate the benefits of the underlying prefetcher.

Upon inferencing, the *Feature Collector* inside PAM gathers all the features, derived from the metadata, and hashes each feature value into their corresponding index to the weight tables. Next, the weights from all the features are accumulated and fed into the activator. If the accumulated weight exceeds the activation threshold, then PAM makes a

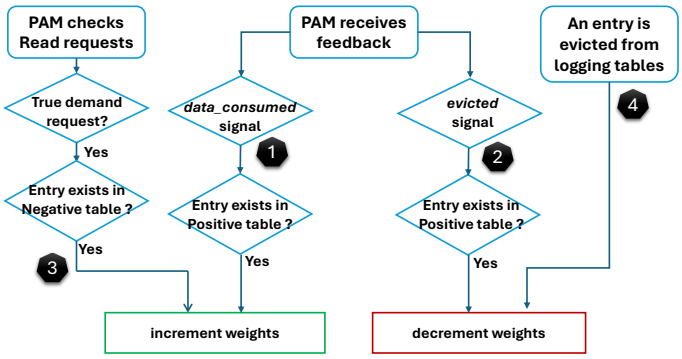

Fig. 3. PAM training flowchart.

positive decision, i.e., the prefetch request is approved and treated as a normal request. Otherwise, PAM rejects the prefetch and discards the request. Once the decision is finalized, the metadata values and the decision are stored in the logging tables, along with any information needed to retrieve the state of this decision, for later usage of at the training stage. The details of the logging tables are discussed in Section 3.6.

3.5 Training

Typical prefetchers rely on the metadata from the cache and the processor to make predictions. The metadata usually also include some feedback information to train the prefetcher, such as *data-consumed*, i.e., prefetched data is consumed by the core; or *evicted*, i.e., a prefetched cache line is evicted from cache without being used. These feedback metadata are now made available to the memory controller, and can be utilized to train and improve PAM's decision-making quality.

There are four training opportunities in PAM: (1) on *data-consumed* signal, (2) on *evicted* signal, (3) on demand request hit in logging table, and (4) on entry eviction from logging tables, as shown in Figure 3. Firstly, training is invoked every time PAM receives feedback from LLC. (1) On a *data-consumed* feedback signal, if the entry is found in the *positive* logging table, PAM invokes training with a true positive outcome, incrementing the weights. This means that the decision previously made by PAM was a correct prediction, which allowed the requested prefetch to receive its data from the off-chip DRAM. (2) On an *evicted* feedback signal, if the entry is found in the *positive* logging table, PAM trains its perceptron weights with a false positive outcome, decrementing the weights. This means the prediction made previously was a wrong decision, and thus the requested data was unnecessarily approved and prefetched to cache.

Secondly, PAM analyzes Read requests coming from LLC for training opportunities. As mentioned in Section 3.4, there are two types of Read requests coming from LLC: a speculative prefetch, and a true demand from the processor. PAM checks every true demand coming to the controller against the *negative* logging table. (3) If an entry is found in the table, it signifies a wrong decision made previously by PAM, which means a needed prefetch request was wrongfully rejected. Training will be invoked in this scenario with a false negative outcome, i.e. incrementing the weights, to prevent future decision from making the same mistake again. Note whenever there is a hit on the logging tables, the entry in the table used in the training stage will be invalidated after the training is done. (4) Lastly, when an entry is evicted from logging tables, PAM also trains itself by decrementing the feature weights accordingly since the corresponding prefetch is deemed unnecessary (more details in Section 3.6).

Once training is invoked, the address of the cache line is used to retrieve the metadata from the logging tables, previously stored at the inference stage. Next, the metadata values are hashed into indices to their corresponding weight tables. The weights are incremented if the outcome is either true positive or false negative, increasing the chance a needed prefetch be accepted next time. Vice versa, the weights are decremented if the outcome is true negative or false positive, increasing the chance an unnecessary prefetch be rejected next time. A saturation check is also implemented at the training stage to ensure no individual weights are saturated, which prevents other non-saturated features from learning. Note that the training stage occurs in the background, as it is not a part of the request's critical path.

3.6 Logging Tables

Since the impact of PAM decision is not immediately known, two logging tables are used to store previous decisions made by PAM. If the prediction is a *positive*, meaning the requested prefetch will proceed as a typical read request, PAM logs the metadata values in the Positive Table. Otherwise, PAM logs the metadata values in the Negative Table. The address of the request is hashed as an index to the table, and a 5-bit tag is used to differentiate every prefetch. The logging tables are 4-way set-associative buffers, with Least-Recently-Used (LRU) replacement policy. The entry in the logging table contains enough information to retrieve each feature and its state when the prediction was made.

The Positive Table contains 1024 entries, while the Negative Table contains 512 entries. Furthermore, PAM trains and improves itself if an entry is evicted from the tables. If an entry is evicted from the Positive Table, it means the approved prefetch data is currently sitting in the cache hierarchy without being used once. MSF treats this as a *bad* prefetch and trains PAM with a false positive outcome, i.e. decrementing the weights. Similarly, if an entry is evicted from the Negative Table, it means that no true demand to the same address has been sent to the memory controller. MSF treats this as a *good* decision since it has successfully prevented cache pollution and unnecessary prefetch. PAM trains itself with a true negative outcome and decrements the weights.

3.7 Offline Features Selection and Threshold Training

Here we discuss the process of selecting features that are used to aid the decision made by MSF, determining whether or not a prefetch request can be accepted at the memory controller. All the features can be derived from the information available in the memory controller or from LLC. We initially come up with 14 single features, as shown in Table 1. While some of these single features are not useful alone, they can be paired up with others to convey more meaningful information. Table 2 shows the features that are used in the offline training stage.

At each training iteration, a random set of five high memory-intensive (HIGH) and five mixed (MIX) workloads are selected, as detailed in Section 4.3. First, MSF is run through the simulations using the datasets with one single feature to collect the accuracy, repeated for all initial set of features. The top-N performing features are *coupled* with another feature on the list to form a set-of-2 combination ($N = 10$ in our experiments). Once the combination is formed, its accuracy is verified via the datasets; any combination that has a lower accuracy than its set-of-1 feature set is eliminated. This ensures the process of combining features can only improve the performance of MSF. Afterwards, the top-N performing set-of-2 features are used in the next iteration to form set-of-3 combinations, while eliminating all combinations that have lower accuracy than their set-of-2 features. This iterative process repeats until there is no improvements on accuracy in the testing phase. Note that in every iteration, we pick a new random dataset consisting of five HIGH and five MIX workloads. The final list of selected program features are shown in Table 3.

Once the program features are finalized, we train MSF to select the optimal activation threshold that is used to make the binary decision. Recall that the weights from all features are accumulated and fed into the activator. PAM

Table 1. Single features and their descriptions.

Feature	Description
1. Rank	Rank address of the request
2. Bank	Bank address of the request
3. Row	Row address of the request
4. Request Address	Address of the request
5. Bank Access (%)	The ratio of a specific bank is accessed out of all available banks
6. Bank Hit Rate (%)	The bank's row buffer hit-rate, binned to 5-bit value
7. Page Hit Rate (%)	The page hit-rate of each core, binned to 6-bit value
8. PC	The Program Counter that triggered
9. Prev-K Addresses	Previous K requests' addresses
10. Prev-K PCs	Previous K requests' Program Counters
11. Row-Recently-Accessed	Binary bit value indicating if a row has been recently accessed
12. Num. Demand Reads in Queue	Number of Demand Read requests enqueued at memory controller
13. Num. Prefetches in Queue	Number of Prefetch requests enqueued at memory controller
14. Prefetcher Accuracy (%)	The underlying prefetcher's accuracy, binned to 6-bit value

Table 2. Initial set of features.

Features	
1. Rank + Bank	2. Request Address
3. Row	4. Bank Access (%)
5. Bank Hit Rate (%)	6. Page Hit Rate (%)
7. Prefetcher Accuracy (%)	8. PC
9. Prev-K PCs	10. Prev-K Addresses
11. Rank + Bank + Bank Access	12. Rank + Bank + Bank Hit Rate
13. Rank + Bank + Page Hit Rate	14. Row + RowRecentlyAccessed
15. Row ⊕ Bank Access	16. Row ⊕ Bank Hit Rate
17. Row ⊕ Page Hit Rate	18. Address ⊕ Bank Access
19. Address ⊕ Bank Hit Rate	20. Address ⊕ Page Hit Rate
21. PC + RowRecentlyAccessed	
22. Num. Demand Reads In Queue + Num. Prefetches In Queue	

Note: The symbol '+' means concatenate, symbol '⊕' means bit-wise XOR.

makes a positive or negative decision if the accumulated weight exceeds the activation threshold or not. Based on our experiments, we choose *act-threshold* to be 16.

3.8 Philosophy of Chosen Features

Here we discuss the chosen program features that help aid PAM in the decision-making. All the features can be derived from the metadata freely-available in the memory controller or from the LLC.

1. Rank + Bank + Bank Hit Rate (%): Every request coming to the Memory Controller will be decoded into its corresponding Rank/Bank/Row number. This feature is computed by concatenating the Rank and Bank number. The resulting bits are then left-shifted and bit-wise OR-ed with Bank Hit Rate. The bank hit rate is a 5-bit binary value that represents the row buffer hit rate of each bank. There are sixteen banks per rank in the DDR4-3200MHz memory modules, and the memory controller keeps a counter of requests going to an open row for each bank and calculates the hit-rate percentage, on a scale of 0 to 100, and bins the result to a 5-bit value. The hit rate calculation is done every 64

Table 3. List of selected features.

Chosen Features	
1. *Rank + Bank + Bank Hit Rate (%)*	2. *Page Hit Rate (%)*
3. *Row ⊕ Bank Access (%)*	4. *Previous-K PCs*
5. *PC + Row Recently Accessed*	6. Prefetcher Accuracy (%)
7. *Row + Row Recently Accessed*	
8. Num. Demand Reads In Queue + Num. Prefetches In Queue	

requests to hide the latency. Note that the counter is reset to zero every 8192 requests to ensure the hit rate reflects the most recent program behavior. Once the counters are reset, the previous epoch's hit rate is used as the replacement until there are sufficient request count (256 requests in our experiments). This feature allows PAM to closely monitor the memory bank-level access locality.

2. Page Hit Rate (%): The memory controller keeps a counter of how many row buffer hits per core, and calculates the hit-rate percentage, on a scale of 0 to 100, and bins it to a 6-bit value. It is binned to 6-bit for more accurate representation of the page hit rate as it is a single feature with no pairing. Similarly to the Bank Hit Rate feature, this calculation is done every 64 requests to hide the latency from critical path, and the counter is reset every 8192 requests. The idea behind this feature is that it allows PAM to correlate the workload spatial locality and determine if a specific prefetch will be beneficial.

3. Row ⊕ Bank Access (%): By itself, the Row number alone does not convey useful information. However, if paired with Bank Access metadata, the feature can provide a different perspective into the memory access pattern. Similar to previous features, the memory controller keeps a counter of how many requests coming in each bank, and calculates the percentage that a specific bank is used out of sixteen total banks. This percentage is binned to a 5-bit value and is XOR-ed with the Row number to inform PAM of any memory access *hotspots*. The percentage calculation is done every 64 requests, and the counter is reset every 8192 requests.

4. Previous-K PCs: This feature is computed by left-shifted-XOR of the previous K Program Counters. The PC is a metadata that is freely available at each cache and prefetcher. Note that we only use 12 bits of PC, as the majority of PC's most significant bits stays the same. The memory controller keeps track of PC for the last K requests from each core. This is done to prevent thrashing between multiple core's execution path. This feature hints PAM of the likelihood that a prefetch is needed based on the current application's execution path. In our experiment, we set $K = 4$.

5. PC + Row Recently Accessed: This feature combines the PC with the *row-recently-accessed* information by replacing the most significant bit of PC with the row-recently-accessed bit. The row-recently-accessed bit value represents whether the request's row has been accessed recently. The memory controller keeps a small 64-entry buffer with *Least-Recently-Used* (LRU) eviction policy, tracking the 64 most recently accessed rows. During inference phase, PAM searches the buffer for a matching row number using 6-bit tag. The row-recently-accessed bit is one if a match is found, zero otherwise. If the entry is not found, the oldest entry is evicted and a new entry with the request's row is inserted. The row-recently-accessed hint value provides a clear view of row's re-usability; and if paired with the request's PC, the feature can help MSF learn about the program's temporal locality.

6. Row Address + Row Recently Accessed: Similar to the *PC + Row Recently Accessed* feature, this feature is computed by replacing the most significant bit of the row address with the *row-recently-accessed* hint bit. This feature allows MSF to determine if a prefetch will be useful or not, based on whether the program is touching a *brand-new* row location or it is simply reusing recent row data.

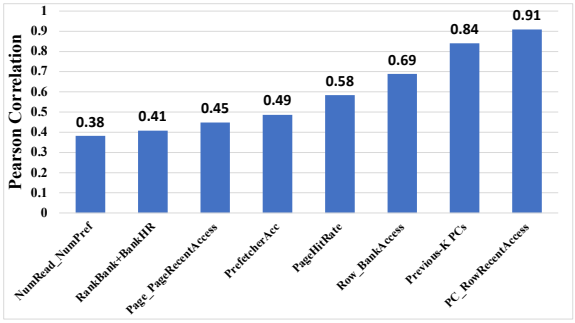

Fig. 4. Pearson Correlation Coefficient for selected features.

7. Prefetcher Accuracy (%): Recall from Section 3.5, MSF relies on the feedback from LLC to train its features. The controller keeps a counter of how many prefetches are needed and how many are evicted from LLC without being used. This information gives an accuracy of the underlying prefetcher housed in each cache. The prefetcher accuracy, ranges from 0 to 100, is binned to 6-bit value and is computed every 64 requests to hide the calculation latency. Note that the counter is reset every 8192 requests to ensure the accuracy represents the most recent program behavior phase, as it changes during the application execution. This feature is particularly useful in predicting a useless cache-polluting prefetch request when the program experiences an erratic data access pattern due to low prefetcher's accuracy.

8. Num. Demand Reads in Queue + Num. Prefetches in Queue: These two separate features are concatenated to give MSF an idea how much bandwidth the underlying prefetcher is consuming compared to the demand requests. Different program phases have different memory access patterns. Some program phases may have higher degree of data locality, resulting in a lower number of Read requests coming to memory controller as most of the data are already in caches; while some other program phases may experience lower degree of data locality, resulting in higher number of cache misses and Read requests coming through to memory. Thus, this feature captures different program behavior correlated with how busy the memory is.

3.9 Pearson Correlation Coefficient

In this section, we analyze the Pearson's Correlation Coefficient (PCC) [27] for all the selected features. PCC measures the linear correlation and its strength between two sets of data. The coefficient ranges from -1 to +1, where a value of 0 indicates no association, and ± 1 indicates strong positive/negative association between the two variables. The statistics are collected and PCC is calculated at the end of each simulation run as the features' weights have settled to their steady values, similar to study in [4].

Figure 4 shows the average Pearson Correlation Coefficients for selected features across all workloads, sorted in an increasing order. The coefficients show the correlation between each future with the prefetching usefulness prediction. Feature 5 (*PC+Row Recently Accessed*) and feature 4 (*Previous-K PCs*) have the highest Pearson coefficients at 0.91 and 0.84, respectively. All the selected features provide moderate to high correlation to the prefetching usefulness prediction.

3.10 Overhead

In this section, we analyze the implementation cost of MSF by showing the storage overhead of its components. Table 4 shows the contents of one entry in the logging tables. For each decision MSF makes, it stores all the metadata values in

Table 4. Entry Contents in Logging Tables.

Feature	Bits
PAM Decision	1
Valid	1
Row Recently Accessed	1
Bank Hit Rate	5
Bank Access	5
Page Hit Rate	6
Prefetcher Accuracy	6
Number of Demand Reads in queue	6
Number of Prefetches in queue	6
Previous-K PCs	12
PC	12
Address	24
Total	85

the logging tables, with each entry of enough information to retrieve the system state when the prediction was made, along with all the features used.

To compute the features in MSF, the memory controller has several counters and buffers to store the necessary information, as shown in Table 5. MSF is designed to support up to sixteen ranks per memory channel, although typical memory systems have significantly fewer number of ranks.

Table 6 shows the total storage overhead of MSF implementation. Note that this overhead is per memory controller, instead of per core as in the PPF scheme. In comparison, PPF consumes 39.34KB of storage overhead per core [4], which equates to 314.72KB of storage in an octa-core system configuration. MSF consumes 29.99KB of storage, significantly less storage overhead since a typical computer system has fewer memory channels than processor cores.

Lastly, MSF requires LLC to send feedback information to the memory controller, as mentioned in Section 3.2. However, these metadata are already accessible in-core and can be retrieved by the memory controller with minimal overhead.

MSF inference is triggered on every memory read requests coming into the controller. However, to avoid additional timing overhead, the inference is removed from critical path of a memory access request. If the inference decision has not been fully computed by the time the memory read request is ready to be served by the memory controller, the request will proceed as a typical request. Based on our experiments, most inference decisions are shorter than the queueing delay in memory Read Queue for memory-intensive workloads. Hence the timing overhead for MSF is negligible.

4 EXPERIMENTAL METHODOLOGIES

4.1 Simulation Configuration

We evaluate our proposed scheme by using cycle accurate x86-64 simulator gem5 [5]. The gem5 simulator is extended to support our scheme MSF. The details of our baseline simulation configurations are shown in Table 7. The DDR4-3200 memory specifications is based on official released white paper by *Micron*® [21]. We model the main memory controller in detail based on [11], with scheduling policy to be first-ready first-come first-serve (FR-FCFS) [30].

Table 5. Counters and Buffers in Memory Controller.

Name	Comment	Bits
Num. Requests per Bank	Used by Bank Hit Rate and Bank Access 13-bit counter, 16 banks/rank, 16 ranks supported	13*16*16
Num. Hits per Bank	Used by Bank Hit Rate 13-bit counter, 16 banks/rank, 16 ranks supported	13*16*16
Num. Hits per Core	Used by Page Hit Rate 13-bit counter, 1 counter/core	13*8
Num. Requests per Core	Used by Page Hit Rate 13-bit counter, 1 counter/core	13*8
Num. Prefetches Used	Used by Prefetch Accuracy	13
Num. Prefetches Evicted	Used by Prefetch Accuracy	13
Row-Recently-Accessed Buffer	64 entries, 6-bit tag per entry, 1 buffer per controller	64*6
Total		7274

Table 6. MSF Overhead.

Structure	Description	Size
Features Weights	Feature 1: 4096 * 5b Feature 2: 512 * 5b Feature 3: 4096 * 5b Feature 4: 4096 * 5b Feature 5: 4096 * 5b Feature 6: 2048 * 5b Feature 7: 512 * 5b Feature 8: 2048 * 5b	13.125 KB
Positive Logging Table	1024 entries * 85b	10.75 KB
Negative Logging Table	512 entries * 85b	5.375 KB
Counters and Buffers	As shown in Table 5	0.887 KB
Previous-K PCs Tracker	4 PCs (12bits each) * 8 cores	0.047 KB
Total		**29.99 KB**

4.2 Metrics

We use several metrics to analyze the performance impact on the underlying prefetcher. The *accuracy* and *coverage* are defined as follows:

$$Accuracy = \frac{Num.\ Useful\ Prefetches}{Num.\ Sent\ Prefetches}$$

$$Coverage = \frac{Num.\ Useful\ Prefetches}{Num.\ Useful\ Prefetches + Num.\ Demand\ Requests}$$

Note that if a prefetch is discarded at the memory controller, it does not count towards the *Num. Sent Prefetches* statistic, as no bandwidth or resources are consumed for that specific prefetch request.

4.3 Workloads

We evaluate our schemes using SPEC CPU2006 and SPEC CPU2017 benchmarks [7]. The workload is categorized based on *misses-per-kilo-instruction* (MPKI), similar to previous studies [29], [25]. Applications with LLC MPKI values higher

Table 7. Major Simulation Parameters.

Parameters	Configurations
Processor	8 out-of-order cores, 4.0 GHz, 8-issue superscalar, x86-64 ISA
Functional Units	6 IntALU, 2 IntMult, 4 FPALU, 2 FPMult per core
IQ, LSQ, and ROB	64 IQ, 32 LQ, 32 SQ, 192 ROB per core
Caches	L1 (P): 32KB Inst/ 32KB Data, 8-way, 64B line, 2 cycles hit latency, 16 MSHRs L2 (P): 256KB, 4-way, 64B line, 10 cycles hit latency, 48 MSHRs L3 (S): 16MB, 8-way, 64B line, 20 cycles hit latency, 64 MSHRs
Memory	1/2 channels, 1/2/4/8 DIMMs/channel, 2 ranks/DIMM, 16 banks/rank
Memory Controller	Queue Size (R/W): 64/64, 20ns overhead
DDR4 Channel Bandwidth	3200MT/s (Mega Transfers/second), 8 bytes/channel, 25.6GB/sec/channel
DDR4 DRAM Latency	DDR4-3200: 22-22-22, tCL 13.75ns, tRCD 13.75ns, tRP 13.75ns

Table 8. Application Category.

Category	Application
Memory-intensive	gcc, bwaves, mcf, milc, zeus, cactusADM, leslie, soplex, GemsFDTD, libquantum, xz, roms, omnetpp, astar, sphinx, parest, fotonik3d, lbm, cactusBSSN
Compute-intensive	perlbench, bzip, gromacs, namd, gobmk, leela, povray, calculix, hmmer, sjeng, h264 tonto, wrf, xalancbmk, x264, blender, imagick, deepsjeng, nab, exchange2, cam4

than ten will be categorized as *memory-intensive* applications. Those with MPKI values lower or equal to ten will be classified as *compute-intensive* applications, as shown in Table 8. Each HIGH workload consists of eight randomly selected memory-intensive applications, LOW workload consists of eight randomly selected compute-intensive applications, while each MIX workload consists of randomly selected four memory-intensive applications and four compute-intensive applications.

Then, we run all available workloads through simulations under baseline configurations with no prefetcher (details in Table 7) to collect their respective MPKI values. Overall, we have 81 workload sets, with 37 memory-intensive (HIGH), 26 compute-intensive (LOW) and 18 mixed (MIX) workloads. The simulations are fast-forwarded to their simulation points, which are produced by SimPoint3.0 [28], then the detailed simulations are run for one billion instructions to collect the statistics.

4.4 Underlying Prefetchers

In our experiments, we implement the following underlying prefetchers in gem5 to show that MSF can work for any generic prefetcher. The underlying prefetcher is deployed at L2/L3 cache, while the Stride prefetcher is deployed at L1 Data cache.

- **Signature Path Prefetcher [18]**: This prefetcher uses compressed history table to accurately predict complex memory access patterns based on its confidence level.
- **AMPM Prefetcher [15]**: The Access Map Pattern Matching (AMPM) prefetcher uses memory access mapping to detect any access pattern from a bitmap-like data structure.

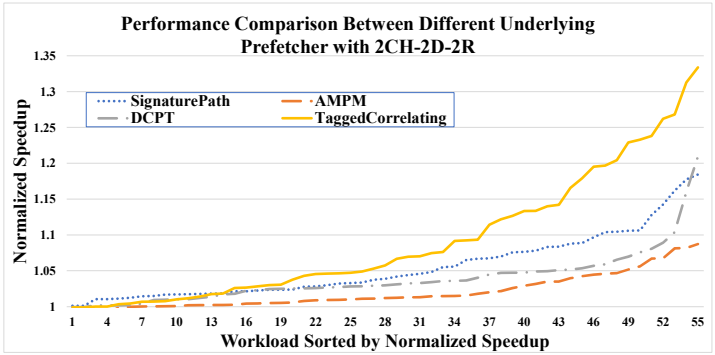

Fig. 5. Performance speedup of MSF under different prefetchers.

- **DCPT Prefetcher [10]**: The Delta Correlating Prediction Tables (DCPT) prefetcher stores the deltas of request misses, which are used to detect patterns in the Delta Correlation matching engine.
- **Tagged-Correlating Prefetcher [13]**: This prefetcher uses tag sequences to correlate the highly-repetitive tag patterns within L1 cache to predict future memory prefetches using pattern history table (PHT) and tag history table (THT).
- **Stride Prefetcher [8]**: Simple prefetcher that predicts the memory access pattern in a stride manner.

4.5 Evaluated Schemes

In our experiments, we evaluate the following related schemes that are implemented in gem5 simulator.

- **Baseline**: The baseline is conventional DDR4-3200 memory system with eight 4.0 GHz out-of-order cores. Each core has private L1 Instruction/Data caches, a private 256KB L2 cache, and a shared L3 cache of size 16MB with underlying prefetchers as shown in Section 4.4. The baseline's memory controller uses FR-FCFS [30] as its scheduling policy, which treats all requests (both demand and prefetch) equally.
- **Perceptron-Based Prefetch Filter (PPF) [4]**: The scheme uses perceptron-learning mechanism to filter unnecessary prefetch requests generated by SignaturePath Prefetcher (SPP). The filter engine uses metadata directly from SPP, such as signature, depth, and confidence, to aid the decision-making. The engine is placed directly inside the prefetcher and only works for SPP specifically.
- **Prefetch-Aware DRAM Controller (PADC) [19]**: The scheme monitors the underlying prefetcher's accuracy to adaptively schedule requests with different priorities. PADC has two components: (1) APS and (2) APD unit. *Adaptive Prefetch Scheduling* (APS) changes the priority of demand/prefetch requests based on the underlying prefetcher accuracy. *Adaptive Prefetch Dropping* (APD) removes prefetch requests from Read Queue based on *age* metric.
- **MSF**: Our proposed memory-side filtering scheme, with details discussed in Section 3.

5 EXPERIMENTAL RESULTS

5.1 Performance with Different Prefetchers

Firstly, we want to show that MSF can perform well under any generic prefetcher, as all the features developed in Section 3.8 can be derived from freely-available metadata within the memory controller or from LLC. Throughout all

experiments, we use xCH-yD-zR to represent the configuration of x memory channels, y DIMMs per memory channel, and z ranks per DIMM.

Figure 5 shows the performance in normalized speedup obtained by MSF under the 2CH-2D-2R configuration with SignaturePath, AMPM, DCPT and Tagged-Correlating as the underlying prefetchers, running HIGH and MIX workload sets. All results have been normalized to the baseline with the corresponding prefetcher enabled but without prefetch filtering. The normalized speedup of each workload by MSF is calculated using the geometric mean of the normalized speedup (relative IPC value) of each application in the workload. All 55 HIGH and MIX workloads have been sorted in increasing order of the normalized speedup of their respective prefetcher.

For memory-intensive (HIGH) workloads, MSF improves the performance by **7.5%** on average (up to 18.4%) with SignaturePath, **3.2%** on average (up to 8.1%) with AMPM, **5.8%** on average (up to 20.9%) with DCPT, and **11.2%** on average (up to 33.4%) with Tagged-Correlating as the underlying prefetchers, respectively. Performance improvement comes mainly from the additional bandwidth and resources that are not used to serve unnecessary prefetch requests. Valuable resources are directed to serving important demand requests and useful prefetches instead.

For mixed (MIX) workloads, MSF performs better by **5.9%** on average (up to 12.8%) than the baseline with SignaturePath, **3.7%** on average (up to 8.7%) with AMPM, **4.5%** on average (up to 10.4%) with DCPT, and **8.0%** on average (up to 23.8%) with Tagged-Correlating prefetchers, respectively. The smaller performance gain than that of memory-intensive workloads is expected since there are fewer generated prefetches and benefit less from filtering out unnecessary prefetches.

Overall, out of all the 55 HIGH and MIX SPEC workloads, MSF improves performance over the baseline on average by **7.0%** with SignaturePath, **3.3%** with AMPM, **5.3%** with DCPT, and **10.5%** with Tagged-Correlating as their underlying prefetchers, respectively. This shows that MSF can work well with various underlying prefetchers.

Next, we want to showcase that MSF has minimal impact on memory access latency in the case of light workloads. We repeat the experiment on 26 compute-intensive (LOW) workloads under the same configurations. Overall, out of 26 LOW workloads, MSF has negligible performance degradation as compared to the baseline with corresponding SignaturePath, AMPM, DCPT, and Tagged-Correlating prefetcher. MSF experiences up to 1.7% performance loss in SignaturePath prefetcher, and negligible performance loss of up to 0.1% in AMPM, DCPT and Tagged-Correlating prefetcher.

5.2 Accuracy, Coverage and Cache Pollution

The main benefit of MSF is that it can filter out unnecessary prefetch requests from being served by the memory system, reducing wasted resources that could have been used in serving other useful requests. Due to space limit, we will use *SignaturePath* as the underlying prefetcher for the following discussions. Using other prefetchers have similar results. Figure 6 and Figure 7 show the accuracy and coverage of SignaturePath as the underlying prefetcher under our scheme. We only show several representative workloads to showcase the impact on accuracy and coverage. The performance gains are made possible by eliminating unnecessary prefetches, which in turns, improves the underlying prefetcher's accuracy. Figure 6 shows the comparison of prefetcher accuracy between the baseline with no filtering scheme, PPF and MSF under SignaturePath prefetcher. Note that PPF scheme only works under SignaturePath prefetcher, while our scheme works for any underlying prefetcher, as discussed in Section 3.1. On average, out of 55 workloads, MSF increases the underlying prefetcher's accuracy by **11.4%** (up to 32.7%) as compared to the baseline without filtering scheme, and **2.2%** (up to 8.4%) compared to the PPF scheme. The MSF Inference engine itself has an accuracy of **83.5%** across all workloads.

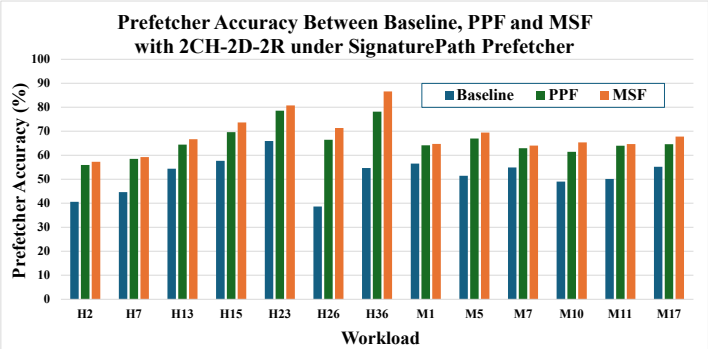

Fig. 6. Comparison of prefetcher accuracy between Baseline, PPF and MSF under SignaturePath prefetcher. Note: H*x* are HIGH workloads, and M*x* are MIX workloads.

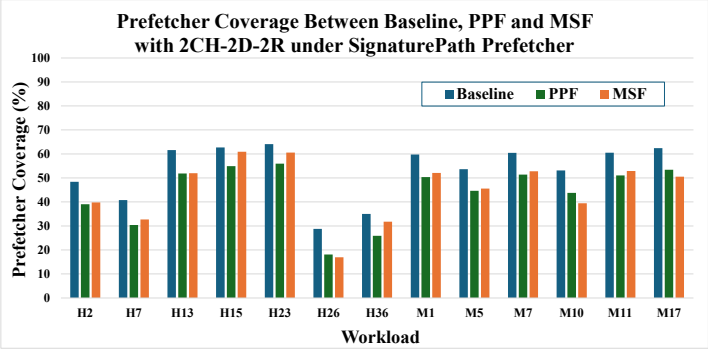

Fig. 7. Comparison of prefetcher coverage between Baseline, PPF and MSF under SignaturePath prefetcher.

MSF does not always make the correct decision. Figure 7 shows the comparison of prefetcher coverage between baseline, PPF and MSF under SignaturePath prefetcher. When MSF makes an incorrect decision, i.e., rejecting a useful prefetch request, it ultimately decreases the coverage. Across all 55 workloads, MSF reduces the prefetcher's coverage by **6.3%** on average (up to 16.5%) as compared to the baseline without filtering scheme. As compared to PPF, MSF improves prefetcher coverage by **1.4%** on average (up to 7.7%). Overall, MSF outperforms PPF in both prefetcher accuracy and coverage.

Furthermore, inaccurate prefetches bring unnecessary data into cache that aren't used by the processor, leading to cache pollution. Figure 8 shows the Last-Level-Cache evicted prefetches without references between baseline, PPF and MSF under *SignaturePath* prefetcher. All the results are normalized to the baseline . We only show representative workloads due to space limitations. On average, out of 55 workloads, MSF reduces the number of LLC evicted prefetches without references by **52.6%**. The reduction in useless prefetches effectively gives more cache space for other demand requests, leading to a **2.7%** decrease of LLC miss rate as compared to the baseline. MSF outperforms PPF in terms of reduction in evicted prefetches without references by **7.2%**, as PPF only manages to reduces 45.4% of useless prefetches as compared to the baseline. Note that MSF also shows similar reduction in LCC evicted prefetches without references with AMPM, DCPT, and Tagged-Correlating prefetcher, while PPF only works under SignaturePath prefetcher.

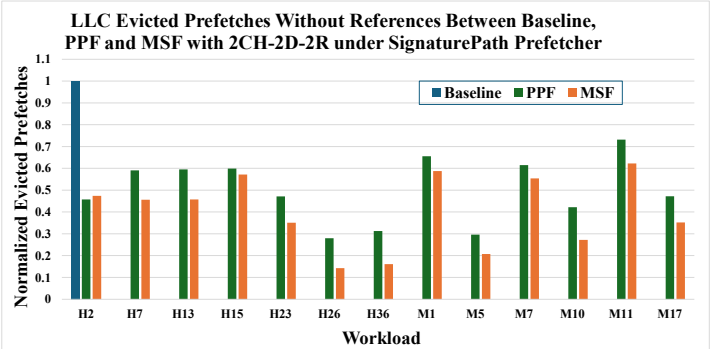

Fig. 8. LLC Evicted Prefetches without references between Baseline, PPF and MSF under SignaturePath prefetcher
Note: The Baseline bars are not shown as all results are normalized to the baseline.

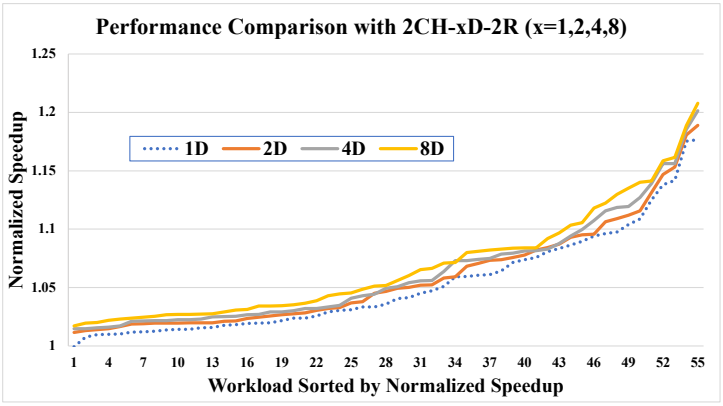

Fig. 9. Performance comparison between MSF and baseline system with various number of DIMMs per channel.

5.3 Scalability of MSF

We also run experiments on the two-channel system with one to eight DIMMs per memory channel to demonstrate the scalability of MSF. Figure 9 shows the performance in normalized speedup obtained by our scheme with SignaturePath prefetcher under one to eight DIMMs per memory channel. All the results have been normalized to their respective 2CH-xD-2R ($x = 1, 2, 4, 8$) baseline configuration, with prefetcher enabled. The workloads have been sorted in increasing order of the normalized speedup. For memory-intensive (HIGH) workloads, the average performance improvement of MSF over the conventional baseline system increases from **7.0%** to **8.7%** as the number of DIMMs per channel grows from one to eight. For mixed (MIX) workloads, the average performance improvement of MSF as the number of DIMMs per channel grows from one to eight increases from **5.2%** to **6.2%** over the baseline. The results indicate that MSF can scale well as the number of DIMMs per channel increases. The increased bandwidth and parallelism introduced by additional DIMMs are utilized in serving more useful prefetch requests as MSF successfully improves the underlying prefetcher's accuracy.

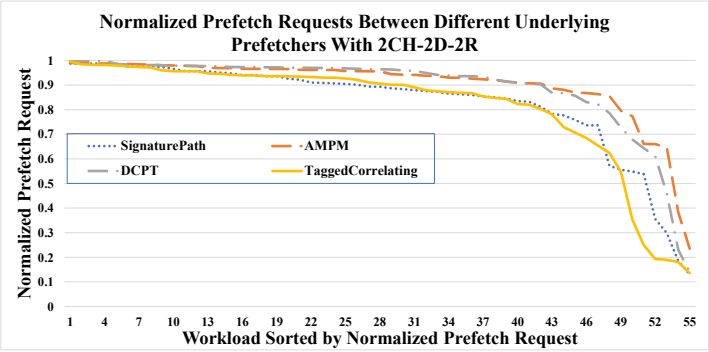

Fig. 10. Number of prefetch requests comparison between MSF and baseline system.

5.4 Energy Consumption

As shown previously, MSF successfully filters out unnecessary prefetches from being served at the memory controller, saving valuable bandwidth and resources. Next, we want to show the energy savings by filtering out unnecessary prefetch requests using our scheme. Figure 10 shows the normalized prefetch requests under the 2CH-2D-2R configuration with various different underlying prefetchers. All the results are normalized to their baseline configurations with its respective underlying prefetcher. The workloads have been sorted by a decreasing order of the normalized prefetch requests. Out of all 55 workloads, on average MSF reduces **17.9%** prefetch requests for SignaturePath, **10.1%** for AMPM, **11.3%** for DCPT, and **19.5%** for Tagged-Correlating prefetchers, respectively. Note that in some workloads, MSF is only able to filter out small percentages of prefetch requests as the underlying prefetcher's accuracy is high.

A direct benefit of filtering out unnecessary prefetches is saving in memory energy consumption, as the memory controller does not waste energy serving unused requests. Figure 11 shows the memory energy consumption breakdown between the baseline and MSF with SignaturePath as the underlying prefetcher for both configurations. We only use several representative workloads to showcase the difference in energy consumption. All the results are normalized to energy consumption of the first HIGH workload (H2) shown in the figure. There are six parts of the energy breakdown: (1) activation, (2) precharge, (3) read, (4) write, (5) refresh, and (6) background. First, compared to that of the baseline, the activation and precharge energy of MSF drops proportionally to the reduced number in requests. This is because the memory controller spends less energy preparing the memory devices for fewer number of requests. Second, the read energy of MSF also drops significantly, because the controller in MSF sends fewer read bursts back to the core. The average read energy reduction is **17.5%** (up to 72.6%), and the overall energy saving is **10.6%** on average (up to 39.3%), as compared to the baseline. The write energy, refresh energy, and background energy stay almost the same as they are independent to the reduction in read requests.

5.5 Performance Comparisons between MSF, PPF and PADC

Figure 12 compares the performance obtained by MSF, PPF, and PADC under the 2CH-2D-2R configuration with SignaturePath as the underlying prefetcher. All the results have been normalized to the baseline with prefetcher enabled. The workloads have been sorted in increasing order of the normalized speedup. For memory-intensive (HIGH) workloads, MSF improves performance by **1.6%** over PPF and by **5.2%** over PADC on average, respectively. For mixed (MIX) workloads, MSF performs better by **0.1%** on average than PPF, and **3.1%** on average than PADC, respectively.

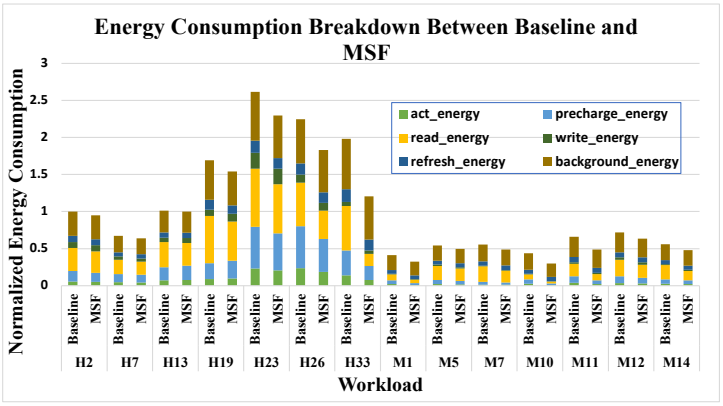

Fig. 11. Memory energy consumption comparison between MSF and baseline system.

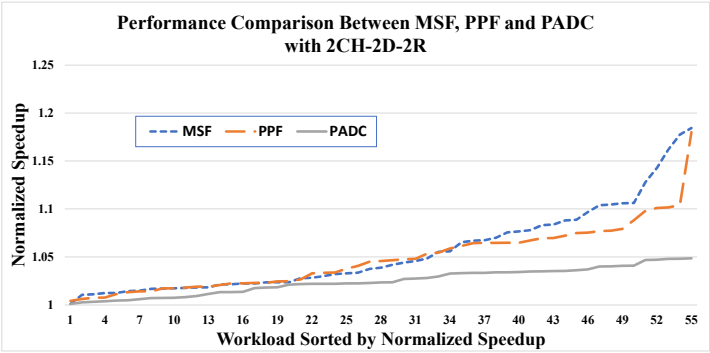

Fig. 12. Performance comparison between MSF, PPF and PADC.

Overall, out of all the 55 workloads, MSF improves performance over PPF scheme by **1.3%**, and **4.6%** over PADC with SignaturePath as their underlying prefetcher. MSF is able to perform better than PPF since MSF also utilizes the memory-side information when making the prefetch filtering decisions. As compared to PADC, MSF is able to improve performance significantly since PADC only utilizes a simple *age* metric to discard bad prefetch requests. Note that PPF scheme only works specifically for SignaturePath prefetcher, as five out of its eight perceptron features are derived directly using SPP's metadata, while MSF works well with any underlying prefetchers as shown in Section 5.1.

6 Conclusions

In this paper, we introduce Memory-Side perceptron-based prefetch Filtering (MSF). It utilizes both core-side and memory-side information and filters unnecessary prefetches based on current core and memory status. The simulation results show that MSF works with any generic underlying prefetcher, as its perceptron features are derived based on metadata available within the memory controller and LLC. MSF improves performance by 7.0% on average and decreases memory energy consumption by 10.6% on average as compared to a baseline configuration with the SignaturePath prefetcher. Our scheme also improves performance over the existing prefetching filtering schemes such as PPF and PADC schemes.

References

[1] Grant Ayers, Heiner Litz, Christos Kozyrakis, and Parthasarathy Ranganathan. 2020. Classifying Memory Access Patterns for Prefetching. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*.

[2] Mohammad Bakhshalipour, Mehran Shakerinava, Pejman Lotfi-Kamran, and Hamid Sarbazi-Azad. 2019. Bingo Spatial Data Prefetcher. In *Proceedings of IEEE International Symposium on High Performance Computer Architecture*.

[3] Rahul Bera, Konstantinos Kanellopoulos, Shankar Balachandran, David Novo, Ataberk Olgun, Mohammad Sadrosadat, and Onur Mutlu. 2022. Hermes: Accelerating long-latency load requests via perceptron-based off-chip load prediction. In *Proceedings of the 55th International Symposium on Microarchitecture*.

[4] Eshan Bhatia, Gino Chacon, Seth Pugsley, Elvira Teran, Paul V. Gratz, and Daniel A. Jiménez. 2019. Perceptron-Based Prefetch Filtering. In *Proceedings of the 46th International Symposium on Computer Architecture*.

[5] N. L. Binkert, R. G. Dreslinski, L. R. Hsu, K. T. Lim, A. G. Saidi, and S. K. Reinhardt. 2006. The M5 Simulator: Modeling Networked Systems. *IEEE Micro* 26, 4 (July 2006), 52–60.

[6] M. J. Charney and T. R. Puzak. 1997. Prefetching and memory system behavior of the SPEC95 benchmark suite. *IBM Journal of Research and Development* 41, 3 (1997), 265–286. doi:10.1147/rd.413.0265

[7] Standard Performance Evaluation Corporation. 2019. SPEC CPU2006 and CPU2017. http://www.spec.org

[8] J.W.C. Fu, J.H. Patel, and B.L. Janssens. 1992. Stride Directed Prefetching In Scalar Processors. In *Proceedings of the 25th International Symposium on Microarchitecture*.

[9] S.I. Gallant. 1990. Perceptron-based learning algorithms. *IEEE Transactions on Neural Networks* 1, 2 (1990), 179–191. doi:10.1109/72.80230

[10] Marius Grannaes, Magnus Jahre, and Lasse Natvig. 2010. Multi-Level Hardware Prefetching Using Low Complexity Delta Correlating Prediction Tables with Partial Matching. In *Proceedings of the 5th International Conference on High Performance Embedded Architectures and Compilers*.

[11] Andreas Hansson, Neha Agarwal, Aasheesh Kolli, Thomas Wenisch, and Aniruddha N. Udipi. 2014. Simulating DRAM controllers for future system architecture exploration. In *Proceedings of IEEE International Symposium on Performance Analysis of Systems and Software*.

[12] Milad Hashemi, Kevin Swersky, Jamie A. Smith, Grant Ayers, Heiner Litz, Jichuan Chang, Christos Kozyrakis, and Parthasarathy Ranganathan. 2018. Learning Memory Access Patterns. In *Proceedings of International Conference on Machine Learning*.

[13] Z. Hu, M. Martonosi, and S. Kaxiras. 2003. TCP: tag correlating prefetchers. In *Proceedings of the 9th International Symposium on High-Performance Computer Architecture*.

[14] Ibrahim Hur and Calvin Lin. 2006. Memory Prefetching Using Adaptive Stream Detection. In *Proceedings of the 39th IEEE/ACM International Symposium on Microarchitecture*.

[15] Yasuo Ishii, Mary Inaba, and Kei Hiraki. 2009. Access map pattern matching for data cache prefetch. In *Proceedings of the 23rd International Conference on Supercomputing*.

[16] D.A. Jimenez and C. Lin. 2001. Dynamic branch prediction with perceptrons. In *Proceedings of the 7th International Symposium on High-Performance Computer Architecture*.

[17] Daniel A. Jiménez and Elvira Teran. 2017. Multiperspective Reuse Prediction. In *Proceedings of the 50th IEEE/ACM International Symposium on Microarchitecture*.

[18] Jinchun Kim, Seth H. Pugsley, Paul V. Gratz, A.L. Narasimha Reddy, Chris Wilkerson, and Zeshan Chishti. 2016. Path confidence based lookahead prefetching. In *Proceedings of the 49th IEEE/ACM International Symposium on Microarchitecture*.

[19] Chang Joo Lee, Onur Mutlu, Veynu Narasiman, and Yale N. Patt. 2008. Prefetch-Aware DRAM Controllers. In *Proceedings of the 41st IEEE/ACM International Symposium on Microarchitecture*.

[20] Heiner Litz, Grant Ayers, and Parthasarathy Ranganathan. 2022. CRISP: critical slice prefetching. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*.

[21] Micron Technology, Inc. 2015. CCMTD-1725822587-9875. https://www.micron.com/-/media/client/global/documents/products/data-sheet/dram/ddr4/8gb_ddr4_sdram.pdf.

[22] Todd C. Mowry, Monica S. Lam, and Anoop Gupta. 1992. Design and Evaluation of a Compiler Algorithm for Prefetching. *SIGPLAN Not.* 27, 9 (sep 1992), 62–73. doi:10.1145/143371.143488

[23] O. Mutlu, H. Kim, D.N. Armstrong, and Y.N. Patt. 2004. Cache filtering techniques to reduce the negative impact of useless speculative memory references on processor performance. In *Proceedings of the 16th Symposium on Computer Architecture and High Performance Computing*.

[24] Onur Mutlu, Hyesoon Kim, David N. Armstrong, and Yale N. Patt. 2005. Using the First-Level Caches as Filters to Reduce the Pollution Caused by Speculative Memory References. *International Journal of Parallel Programming* 33 (2005), 529–559.

[25] Agustín Navarro-Torres, Jesús Alastruey-Benedé, Pablo Ibáñez-Marín, and Víctor Viñals-Yúfera. 2019. Memory hierarchy characterization of SPEC CPU2006 and SPEC CPU2017 on the Intel Xeon Skylake-SP. *PLoS ONE* 14 (2019). https://api.semanticscholar.org/CorpusID:199382452

[26] Agustín Navarro-Torres, Biswabandan Panda, Jesús Alastruey-Benedé, Pablo Ibáñez, Víctor Viñals-Yúfera, and Alberto Ros. 2022. Berti: an Accurate Local-Delta Data Prefetcher. In *Proceedings of the 55th IEEE/ACM International Symposium on Microarchitecture*.

[27] Karl Pearson. [n. d.]. Note on regression and inheritance in the case of two parents. *Proceedings of the Royal Society of London* 58 ([n. d.]), 240 – 242. https://api.semanticscholar.org/CorpusID:121644161

[28] Erez Perelman, Greg Hamerly, Michael Van Biesbrouck, Timothy Sherwood, and Brad Calder. 2003. Using SimPoint for Accurate and Efficient Simulation. In *Proceedings of SIGMETRICS*.

[29] Muhammad M. Rafique and Zhichun Zhu. 2019. FAPS-3D: Feedback-Directed Adaptive Page Management Scheme for 3D-Stacked DRAM. In *Proceedings of the International Symposium on Memory Systems*.

[30] Scott Rixner, William J. Dally, Ujval J. Kapasi, Peter Mattson, and John D. Owens. 2000. Memory Access Scheduling. In *Proceedings of the 27th International Symposium on Computer Architecture*.

[31] Santhosh Srinath, Onur Mutlu, Hyesoon Kim, and Yale N. Patt. 2007. Feedback Directed Prefetching: Improving the Performance and Bandwidth-Efficiency of Hardware Prefetchers. In *Proceedings of the 13th International Symposium on High Performance Computer Architecture*.

[32] David Tarjan and Kevin Skadron. 2005. Merging Path and Gshare Indexing in Perceptron Branch Prediction. *ACM Trans. Archit. Code Optim.* 2, 3 (sep 2005), 280–300. doi:10.1145/1089008.1089011

[33] Elvira Teran, Zhe Wang, and Daniel A. Jiménez. 2016. Perceptron learning for reuse prediction. In *Proceedings of the 49th IEEE/ACM International Symposium on Microarchitecture*.

[34] Haoyuan Wang and Zhiwei Luo. 2017. Data Cache Prefetching with Perceptron Learning. *ArXiv* (12 2017).

[35] Zhenlin Wang, K.S. McKinley, A.L. Rosenberg, and C.C. Weems. 2002. Using the compiler to improve cache replacement decisions. In *Proceedings of International Conference on Parallel Architectures and Compilation Techniques*.

[36] Carole-Jean Wu, Aamer Jaleel, Margaret Martonosi, Simon C. Steely, and Joel Emer. 2011. PACMan: Prefetch-Aware Cache Management for High Performance Caching. In *Proceedings of the 44th IEEE/ACM International Symposium on Microarchitecture*.

[37] Xiangyao Yu, Christopher J. Hughes, Nadathur Satish, and Srinivas Devadas. 2015. IMP: Indirect memory prefetcher. In *Proceedings of the 48th IEEE/ACM International Symposium on Microarchitecture*.

[38] Xiaotong Zhuang and Hsien-Hsin Lee. 2007. Reducing Cache Pollution via Dynamic Data Prefetch Filtering. *Computers, IEEE Transactions on* 56 (Feb. 2007), 18–31. doi:10.1109/TC.2007.250620

Received 09 July 2025